Validator-Guided Repair on a Shared Initial LLM Candidate: A Preregistered Paired Evaluation for Network Configuration Safety

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired confirmatory experiment (PREREG-CAND-001-VER-GVR-2026-09-01) to measure whether permitting a deterministic network-configuration validator plus one automated repair changes the rate of validator-valid executable configurations relative to leaving the identical sampled LLM candidate unguarded (shared_initial_candidate semantics). Using the declared generator gpt-5-mini (hosted adapter openai_responses) and deterministic_netops_validator_v5 on N = 200 paired intents (completed_episode_count = 400; model_calls_used = 200), every shared initial candidate passed validator checks and no repairs were attempted or applied (repair_attempt_count = 0; repair_applied_count = 0). Observed outcomes: baseline_success = 200/200, guarded_success = 200/200; paired contingency n11 = 200, n10 = 0, n01 = 0, n00 = 0; exact McNemar two-sided p = 1.0; paired bootstrap percentile 95% CI (seed = 1729; 10,000 resamples) = [0.0, 0.0]. We present artifact-grounded methodology, execution accounting, representative validator diagnostics, compact contingency and repair summaries, and a focused discussion clarifying the causal limits of the executed estimand under shared_initial_candidate semantics and preregistered follow-on designs required to measure repair efficacy under independent sampling, higher difficulty, or adversarial inputs.**

I. Introduction

Generative large language models (LLMs) are increasingly proposed to translate operator intent into device configuration sequences in network operations (NetOps). Erroneous sequences, syntactic mistakes, or fabricated fields can cause service disruption or policy violations when applied to devices. A common architectural mitigation is a generate→validate→repair (GVR) pipeline: generate candidate configuration(s), deterministically validate them against intent/topology/workflow constraints, and trigger automated repair when the validator finds faults. Prior work documents verifier-guided improvements in infrastructure-as-code and related code-generation domains and recommends grounding strategies to reduce hallucination in operational tasks [1–3]. However, whether verifier-guided pipelines reduce execution-level operational risk for network configurations remains an open empirical question because prior demonstrations often measure textual correctness or IaC test suites rather than executed device-state safety in packet-level or emulator contexts [3,4].

This paper reports a fully preregistered paired confirmatory experiment that isolates the downstream causal contribution of a deterministic validator and any permitted repair acting on the exact same sampled LLM candidate (shared_initial_candidate semantics). Under these semantics baseline and guarded episodes for each intent begin from the identical initial generation; any paired difference can therefore only arise from deterministic validation and a permitted repair of that same candidate. Our primary research question is: under shared_initial_candidate semantics, does permitting a deterministic validator plus at most one automated repair change the proportion of validator-valid executable configurations relative to leaving the same initial candidate unguarded? The contribution is an artifact-grounded report of the executed estimand with complete execution accounting, exact inferential statistics, representative validator diagnostics, and precise recommendations for preregistered follow-on designs that can measure repair efficacy when independent sampling, elevated difficulty, or adversarial inputs are employed.

II. Related Work

Hallucination and factual unreliability in LLM outputs are well documented and motivate grounding, retrieval, and verification strategies for operational tasks [1]. Retrieval-augmented and evidence-grounded pipelines can reduce hallucination in operations-style QA, though these works evaluate textual or documentation correctness rather than device-level executability [2]. Generate→verify→repair architectures have shown empirical improvements in IaC and program generation (for example, verifier-guided fine-tuning and policy-guided repair loops) and are a primary methodological inspiration for our work [3]. Recent NetOps evaluations examine intent translation and LLM capability but emphasize capability benchmarks rather than standardized execution-level safety measures that map generated commands to executable device states in a simulator/emulator [4]. Comprehensive AIOps surveys highlight the need for execution-aware benchmarks and validator fidelity uplift to bridge generation to operational safety [5].

From these verified sources we identified two methodological gaps that motivated the current paired design. First, verifier-guided demonstrations in adjacent domains do not directly measure whether corrections prevent unsafe executable states when enacted on networks; empirical transferability to NetOps therefore requires execution-level validation. Second, many prior studies mix sampling variability with downstream validator/repair effects; precisely defined estimands that separate these sources of variation are uncommon. We therefore

preregistered a shared_initial_candidate paired design that isolates the downstream validator/repair effect on identical generator outputs.

III. Methodology

Preregistration, estimand, and paired semantics. We preregistered the study as PREREG-CAND-001-VER-GVR-2026-09-01 and froze the primary estimand before any model calls. Primary estimand: paired_success_rate_difference_guarded_minus_baseline (the per-intent paired difference in the proportion of validator-valid executable configurations). The design is a confirmatory paired binary comparison over task_count = 200 intents producing planned_episode_count = 400 episodes. Crucially, generation semantics were registered and executed as shared_initial_candidate: exactly one sampled initial generation per pair was produced and deterministically reused by both baseline and guarded episodes. This design choice isolates any downstream causal effect to deterministic validation and the permitted repair acting on the identical candidate; it intentionally removes sampling variability between conditions. We emphasize that the archived model_configuration.json records temperature = null/unset; null/unset is not evidence of deterministic sampling and does not imply temperature = 0.

Generation and provider audit. The declared generator used in execution was gpt-5-mini via the hosted adapter openai_responses. The execution manifest records model_calls_used = 200 (one shared initial generation per pair), hosted_model_call_event_count = 200, completed_hosted_model_call_event_count = 200, failed_hosted_model_call_event_count = 0, cache_miss_count = 200, and stage_counts indicating shared_initial_generation = 200. These provider-audit counts confirm that the adapter produced a single sampled candidate per pair for reuse by both arms as preregistered. The experiment permitted no retrieval augmentation (RAG=false in the preregistration) and the adapter enforced the maximum_model_calls limit in the preregistration.

Task generation, contamination controls, and task manifest. A deterministic task generator produced the 200 synthetic intent_configuration_repair_v1 tasks. Each manifest entry includes initial_state, expected_final_state, required_sequence, difficulty annotations (e.g., level tags: very_hard, extreme), allowed_touched_fields, and a generation_seed to permit exact replay. Preexecution contamination controls enforced an exact-match SHA-256 policy and high-similarity n-gram checks (n-gram Jaccard threshold, preregistered high-similarity flag threshold = 0.9). No exact-match contamination exclusions were flagged and no confirmatory exclusions were required; any flagged items would have been moved to an exploratory leaked bucket per the contamination_plan. Contamination artifacts and the fixed task manifest are archived for audit and replay.

Deterministic validator, repair policy, and scored outcome mapping. Scoring used deterministic_netops_validator_v5 with scoring_rule = "full_intent_constraint_validation." The validator is deterministic and structured: it parses candidate command sequences into normalized_configuration, compares parsed_command_count to task required_command_count, enforces workflow_policies (final_group_constraints, minimum_active_in_groups, must_be_down_for_fields, protected_interfaces), evaluates dependency rules, and emits structured validator_trace objects including validation_before and validation_after subtables, violation_codes, violation_count, parsed_command_count, required_command_count, observed_touched_field_count, and a repair_applied boolean. For guarded episodes a single automated repair attempt was permitted (maximum_repair_calls_per_task = 1); the repair module, if invoked, would generate an edited candidate to be revalidated. Under shared_initial_candidate semantics the baseline arm deterministically reused the same sampled candidate without repair; only the guarded arm could alter that identical candidate by invoking the permitted repair. For analysis, validator.valid == true mapped to a binary "executable/valid" outcome per episode; validator.valid == false would trigger repair_attempt logic in guarded episodes.

Missingness, exclusions, and analysis procedures. The preregistration prescribed the complete_pair_primary missingness policy: exclude incomplete pairs from the primary confirmatory McNemar test. A sensitivity analysis (failure-as-zero) was preregistered to treat any missing or failed episode as unsafe. The primary hypothesis used an exact two-sided McNemar test on paired binary outcomes; paired bootstrap percentile CIs were computed (bootstrap_seed = 1729; bootstrap_resamples = 10,000) to quantify uncertainty in the paired difference. Exploratory subgroup comparisons by difficulty pattern and failure category were labeled exploratory and would be adjusted using Benjamini–Hochberg FDR = 0.05 when reported. Multiplicity for the main confirmatory test was controlled by design (single prespecified primary test).

Execution reconciliation and reproducibility artifacts. Execution completed without infrastructure failures and matched preregistered accounting: completed_episode_count = 400, failed_episode_count = 0, complete_pairs = 200. Deterministic reconciliation checks verified pair accounting, marginal consistency, and provider audit stages. Per-episode scoring JSON artifacts record the shared_initial_candidate, raw_response_sha256, normalized_response, the validator_trace object, repair_applied flags, and the scorer_id/version. Representative artifact excerpts (archived) demonstrate the validator fields used to compute binary outcomes (e.g., parsed_command_count, required_command_count, normalized_configuration, violation_count). All artifacts required to replay the executed estimand (task manifest, responses, scoring traces, analysis CSVs, and model configuration) are archived in the evidence bundle to permit deterministic replay of the exact shared_initial_candidate sampling and subsequent scoring.

Implementation notes and limitations of methods. The validator performs static, deterministic checks against task

workflow_policies and expected command sequences; it does not perform vendor CLI execution nor packet-level emulation. This design choice was deliberate to isolate syntactic and workflow-policy correctness; it is a recognized construct validity limitation for dynamic runtime behaviors. The preregistered repair policy limited automated repairs to one attempt per guarded episode to bound repair budget and analysis interpretation. Finally, the model_configuration records temperature = null/unset; the methods explicitly note that null/unset is not evidence of deterministic sampling and that sampling nondeterminism may still have occurred during the single sampled generation that was archived and deterministically reused.

IV. Results

Execution accounting and deterministic reconciliation. Execution completed per the manifest: planned_episode_count = 400, completed_episode_count = 400, failed_episode_count = 0, model_calls_used = 200 (execution/manifests archived). Analysis missingness_summary reports complete_pairs = 200 and zero failed or unscorable episodes. Deterministic reconciliation shows stage_counts = {shared_initial_generation: 200} and cross_condition_cache_key_reuse_observed = false; all deterministic consistency checks passed.

Primary outcomes and contingency. Condition summaries: baseline_successes = 200/200 and guarded_successes = 200/200 (success_rate = 1.0 each). Paired contingency: n11 = 200, n10 = 0, n01 = 0, n00 = 0. Exact McNemar two-sided p = 1.0. Paired bootstrap (seed = 1729; 10,000 resamples) percentile 95% CI for the paired difference = [0.0, 0.0]. These artifacts are archived as analysis/condition_summary.csv and analysis/paired_contingency_table.csv.

Repair activity and validator diagnostics. Repair_attempt_count = 0 and repair_applied_count = 0 across the confirmatory sample: the validator flagged no violations on any shared initial candidate and therefore the permitted guarded repair path was never invoked. Representative per-episode JSON artifacts (examples archived under execution/scoring/, e.g., task-000004-baseline.json and task-000004-guarded.json) show identical shared_initial_candidate content and identical validation_after.normalized_configuration values. Example shared initial candidate (task-000004):

interface edge2 admin down interface edge2 vlan 40 interface edge2 admin up interface edge1 admin down

For that example parsed_command_count = 4, required_command_count = 4, and validator.valid = true. Similar diagnostics hold for sampled representative tasks (task-000005, task-000006).

Contamination and provider audit summary. Preexecution contamination checks flagged no exact matches; analysis/contamination_summary.csv is empty. Provider audit recorded hosted_model_call_event_count = 200 and cache_miss_count = 200. No infrastructure failures or retrials occurred; analysis/analysis_log.jsonl documents the bootstrap settings used (bootstrap_resamples = 10000; bootstrap_seed = 1729) and that paired analysis completed successfully.

Compact repair summary (explicit). Table 2 (below) provides the preregistered reviewer request summary derived from archived artifacts: total_pairs = 200; repair_attempt_count = 0; repair_applied_count = 0. The absence of any validator detections fully explains the observed degenerate contingency: with no validator detections, the guarded repair path had no opportunity to alter outcomes.

V. Discussion

Interpreting the executed estimand. The shared_initial_candidate semantics were intentionally chosen to isolate the causal effect of deterministic validation and any permitted repair acting on the exact same generator output. The executed estimand therefore answers a narrowly defined causal question: when baseline and guarded episodes begin from the identical sampled candidate, can deterministic validation and at most one automated repair change the validator-valid/executable outcome? The observed null effect (paired difference = 0.0) shows that, for this task bank and this single sampled candidate per pair, the validator found no violations and the permitted repair path was never exercised. Importantly, this does not evaluate whether guarded pipelines reduce first-pass generator error rates under independent sampling or different model temperatures; such claims would require a different preregistered estimand and execution.

Threats to validity (artifact-grounded). Internal validity: the paired shared_initial_candidate design eliminates sampling variability across arms and thus preserves attribution of any observed repair effect to validator/repair alone; however, this design precludes measurement of generator-level effects. Construct validity: deterministic_netops_validator_v5 enforces syntactic and workflow-policy checks and maps results to a boolean valid flag; it is an abstraction that does not execute vendor CLI on physical hardware nor perform packet-level emulation—dynamic protocol/timing effects are therefore not represented. External validity: results are conditioned on one declared generator (gpt-5-mini via adapter openai_responses), a synthetic deterministic task bank, and one validator implementation; generalization beyond these artifacts is unsupported. Conclusion validity: zero discordant outcomes preclude inference about repair benefit magnitude under different sampling regimes because the guarded path was not invoked.

Operational implications and preregistered follow-on designs. The null result here should not be read as evidence that GVR pipelines are unnecessary in practice. Instead it demonstrates an estimand boundary: under shared_initial_candidate semantics and the executed task bank the guarded pipeline had no opportunity to change outcomes. To empirically evaluate repair efficacy in operationally relevant regimes, we preregistered and recommend the following estimand/design modifications (each reuses archived artifacts where applicable): (A) remove shared_initial_candidate semantics so baseline and guarded arms receive independent first-pass samples (this permits the repair path to act on generator faults but requires

a revised paired/unpaired analysis plan to separate generation variability from repair effects); (B) select or synthesize tasks labeled "extreme" difficulty or inject adversarial perturbations to raise expected validator failure rates and thereby exercise repair logic; (C) preregister non-null temperature values and temperature sweeps to evaluate sample diversity versus validator-failure tradeoffs; (D) increase validator fidelity by integrating vendor CLI semantics or packet-level emulation so repairs are evaluated against dynamic runtime behavior; (E) expand to multi-model sampling to assess generality across generator families.

Reproducibility and artifact utility. All per-episode scoring JSON artifacts include validator traces with validation_before/validation_after subtables, normalized_configuration, parsed_command_count, required_command_count, and repair_applied flags permitting independent verification that repairs did not run. Execution and analysis artifacts (execution/raw_results.jsonl, execution/task_manifest.jsonl, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/analysis_log.jsonl) are archived in the evidence bundle and enable replay of the exact shared_initial_candidate sampling. The model_configuration.json records declared_model_version = "gpt-5-mini" and temperature = null; null/unset must not be interpreted as deterministic sampling. The master prompt immutable reference is archived (see Disclosure). These artifacts are sufficient to reproduce the executed estimand and deterministic reconciliation.

VI. Limitations

- Shared_initial_candidate semantics: the design produced exactly one sampled initial generation per intent and reused it across baseline and guarded conditions; this measures validator/repair effects only and cannot detect differences caused by independent per-condition sampling or prompt engineering.
- Temperature unset (temperature = null) in the archived model configuration: null/unset is not evidence of deterministic sampling and does not imply temperature = 0; sampling nondeterminism may still have occurred during the single sampled generation.
- Single model and single hosted provider: results reflect gpt-5-mini under the archived adapter; generalization to other models or model families is unsupported by these artifacts.
- Synthetic task bank and validator abstraction: tasks were synthetic 'intent_configuration_repair_v1' items and the deterministic validator is an abstraction; realistic vendor CLI idiosyncrasies, emergent runtime interactions, and hardware effects were not executed on live devices.
- No observed validator failures: all initial candidates were validator-valid, so the experiment could not assess the repair step's effectiveness; the null effect therefore reflects the task bank and sampling semantics rather than evidence that GVR is unnecessary in general.

VII. Conclusion

We executed a fully preregistered paired confirmatory experiment that measures the downstream effect of a deterministic validator plus an allowed single automated repair on the exact same sampled LLM candidate per intent (shared_initial_candidate semantics). Using the declared generator gpt-5-mini and deterministic_netops_validator_v5 on 200 paired intents, every shared initial candidate passed validation and no repairs were attempted or applied; guarded and baseline outcomes were identical for the executed estimand (paired difference = 0.0; McNemar p = 1.0; paired bootstrap CI = [0.0, 0.0]). This artifact-grounded null result demonstrates an estimand boundary: under shared_initial_candidate semantics and the executed task bank the guarded pipeline could not be exercised and therefore could not change outcomes. It does not imply that GVR pipelines are unnecessary generally. Preregistered follow-on experiments that (1) remove shared_initial_candidate semantics, (2) increase task difficulty or inject adversarial inputs, (3) set explicit sampling parameters, and (4) raise validator fidelity are required to empirically evaluate repair efficacy under realistic sampling variability and adversarial conditions. The archived artifacts and reconciliation files enable exact reproduction of the executed estimand and provide a secure base for precise preregistered extensions.

References [1] A. Alansari and H. Luqman, "Large Language Models Hallucination: A Comprehensive Survey," arXiv preprint arXiv:2510.06265, 2025. [2] B. Zhang, H. Rao, and D. Zhao, "Evidence-Grounded RAG for Cloud-Native DevOps: Hallucination-Resistant AIOps Question Answering over Private Operations Documents," J. Autom. Cloud-Native Syst., 2024. [3] P. Jana et al., "TerraFormer: Automated Infrastructure-as-Code with LLMs Fine-Tuned via Policy-Guided Verifier Feedback," Proc. ACM Symp. (2026). [4] Y. Miao et al., "An Empirical Study of NetOps Capability of Pre-Trained Large Language Models," arXiv:2309.05557, 2023. [5] L. Zhang et al., "A Survey of AIOps in the Era of Large Language Models," ACM Comput. Surv., 2025.

One-line reiteration of the primary estimand limit: the preregistered primary estimand intentionally used shared_initial_candidate semantics; evaluating per-condition independent sampling requires a different preregistration and analysis plan (explicitly recommended above).

Disclosure Statement

Disclosure (concise): The human Research Director selected the broad topic family (Generative AI / LLMs for NetOps) before the autonomy lock. After the autonomy lock the autonomous system performed literature review, experiment selection, preregistration (PREREG-CAND-001-VER-GVR-2026-09-01), code generation, execution, deterministic validation, automated analysis, and manuscript drafting without manual human editing of technical content. Declared model used in execution: gpt-5-mini via hosted adapter 'openai_responses' (execution used model_calls_used = 200; archived model_configuration.json records temperature = null/unset;

null/unset is not evidence of deterministic sampling and does not imply temperature = 0). Generation semantics executed: shared_initial_candidate — exactly one sampled initial generation per intent was produced and deterministically reused for both baseline and guarded conditions. Validator: deterministic_netops_validator_v5 scored candidates using 'full_intent_constraint_validation' and a single automated repair iteration was permitted in guarded episodes; in this run the validator flagged no violations and no repairs were applied (repair_attempt_count = 0; repair_applied_count = 0). Execution accounting: completed_episode_count = 400 (200 paired intents) completed without preregistration deviations. Master prompt immutable reference: provenance/master_prompt.txt — SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196 add77f5acdf1582bb39b15ba. Pre-lock infrastructure development and rehearsal occurred but pre-lock artifacts were not used as empirical evidence in this final run. After the autonomy lock no human manually rewrote technical results, manuscript text, figures, or references.

REFERENCES

[1] Aisha Alansari, Hamzah Luqman, "Large Language Models Hallucination: A Comprehensive Survey", 2025, 10.48550/arxiv.2510.06265

[2] Boning Zhang, Hengning Rao, Derek Zhao , "Evidence-Grounded RAG for Cloud-Native DevOps: Hallucination-Resistant AIOps Question Answering over Private Operations Documents", 2024, 10.69987/jacs.2024.40308

[3] Prithwish Jana, Sam Davidson, Bhavana Bhasker, Andrey Kan, Anoop Deoras, Laurent Callot, "TerraFormer: Automated Infrastructure-as-Code with LLMs Fine-Tuned via Policy-Guided Verifier Feedback", 2026, 10.1145/3786583.3786898

[4] Yukai Miao, Yu Bai, Li Chen, Dan Li, Haifeng Sun, Xizheng Wang, Ziqiu Luo, Ren, Yanyu, Dapeng Sun, Xiuting Xu, Qi Zhang, Chao Xiang, Xinchi Li, "An Empirical Study of NetOps Capability of Pre-Trained Large Language Models", 2023, 10.48550/arxiv.2309.05557

[5] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635